

Learn Low-Level Design in 25 Days: Day 20—API Design Principles in LLD

(Part of the series: Low-Level Design in 25 Days)

TechPreneur

Published November 1, 2025

Free: No

Read on Freedium · original

Contents

The Unseen Interface Behind Every Great System 0

Core Concepts — Designing APIs That Don't Hurt to Use 0

1. What Makes a Good API? 0

2. Principles that Shape Great APIs 0

a) Consistency 0

b) Statelessness 0

c) Versioning 0

d) Error Handling 0

e) Pagination & Filtering 0

3. Example: Designing a Food Delivery API 0

Requirements: 0

4. Internal API Design (Beyond HTTP) 0

Visual Overview 0

Try It Yourself 0

Summary — What We Learned Today 0

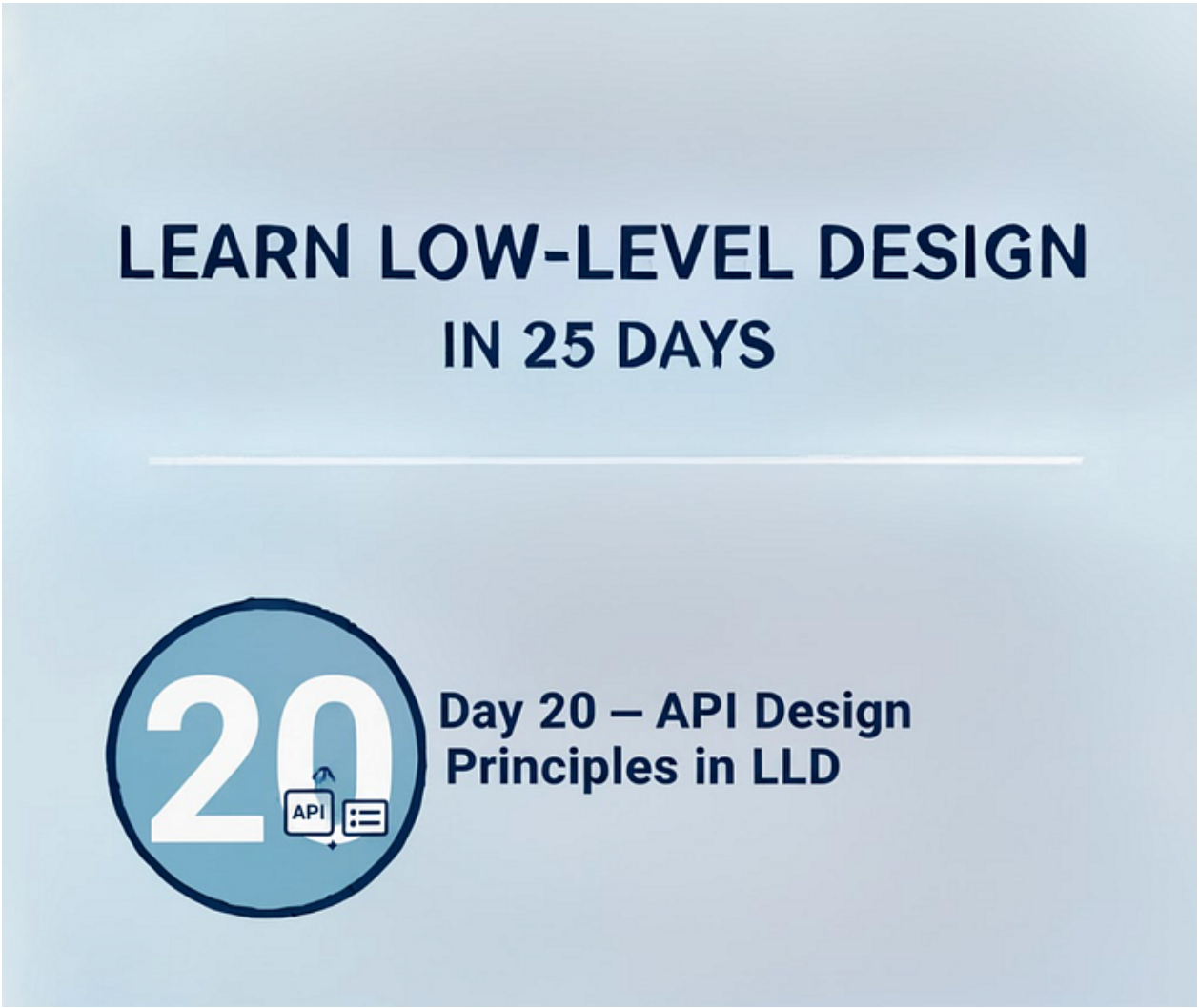
Coming Up Next 0

The Unseen Interface Behind Every Great System

You've probably hit an API that made you mutter, "Who designed this thing?" Maybe it returned 400 Bad Request for no apparent reason. Or maybe its documentation read like a riddle.

Now think about the APIs you love — clean, intuitive, predictable. They just make sense. Stripe, GitHub, and Twilio are often called out not just for their features, but for how beautifully their APIs are designed.

Welcome to **Day 20 of Learn Low-Level Design in 25 Days** — today, we're diving into **API Design Principles**. Whether you're building REST endpoints, SDK interfaces, or service contracts between microservices, great API design is what separates scalable, maintainable systems from spaghetti chaos.



By the end of this lesson, you'll understand how to design APIs that are:

- **Consistent** across modules
- **Predictable** for clients

- **Resilient** to change
- **Easy to evolve** as your product grows

Core Concepts — Designing APIs That Don't Hurt to Use

Think of an API as a contract between your system and the world. It defines what others can do, not how you do it. Your goal as a designer is to make that contract clear, consistent, and hard to misuse.

Let's break it down.

1. What Makes a Good API?

A good API is like a good restaurant menu:

- It's clear what's available.
- The naming is intuitive.
- You know what you'll get before ordering.

In technical terms, that means:

- **Clear naming**: `createOrder()` is better than `processData()`.
- **Consistent structure**: Endpoints follow a predictable pattern (`/orders`, `/orders/{id}`).
- **Proper abstraction**: You expose what can be done, not how it's implemented.

Example — Bad API design:

```
POST /doAction
{
  "type": "createOrder",
  "payload": { ... }
}
```

This is a nightmare. The client has to know the "type" and internal meanings. It's not RESTful or intuitive.

Better:

```
POST /orders
{
  "items": [...],
  "address": {...}
}
```

The intent is clear. You're creating an order — not "doing some action."

2. Principles that Shape Great APIs

Let's explore the pillars that define solid API design.

a) Consistency

Every endpoint, method, or function should behave in a predictable way. If `GET /users` returns a list, then `GET /orders` should too — not a random object.

b) Statelessness

Especially for REST APIs — no hidden state between requests. Every call is independent. That's how systems like Swiggy or Uber scale horizontally across thousands of servers.

c) Versioning

Never break clients silently. If you need to change your API contract, version it:

```
/api/v1/orders
/api/v2/orders
```

Backward compatibility is your users' sanity.

d) Error Handling

A clear error model saves days of debugging.

```
{
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "No order found with id 123"
  }
}
```

That's infinitely better than `"Something went wrong."`

e) Pagination & Filtering

Avoid dumping thousands of records. Use:

```
GET /orders?limit=10&offset=20
```

Clients (and your servers) will thank you.

3. Example: Designing a Food Delivery API

Let's say we're designing the core of a **Food Delivery App** (from Day 19).

Requirements:

- List restaurants
- Place an order
- Track order status

Entities

```
Restaurant
Order
User
DeliveryPartner
```

Endpoints

```
GET /restaurants                → list all restaurants
GET /restaurants/{id}           → get restaurant details
POST /orders                    → create a new order
GET /orders/{id}                → fetch order status
PATCH /orders/{id}/cancel       → cancel order
```

That's clean and self-explanatory. Each resource (noun) has clear actions (verbs). This is the foundation of good REST design.

4. Internal API Design (Beyond HTTP)

Not all APIs are exposed over HTTP. Inside large systems, we define **service interfaces** between modules.

Example — a `PaymentService` interface:

```
public interface PaymentService {
    PaymentResponse initiatePayment(Order order);
    PaymentStatus getPaymentStatus(String paymentId);
    void refund(String paymentId);
}
```

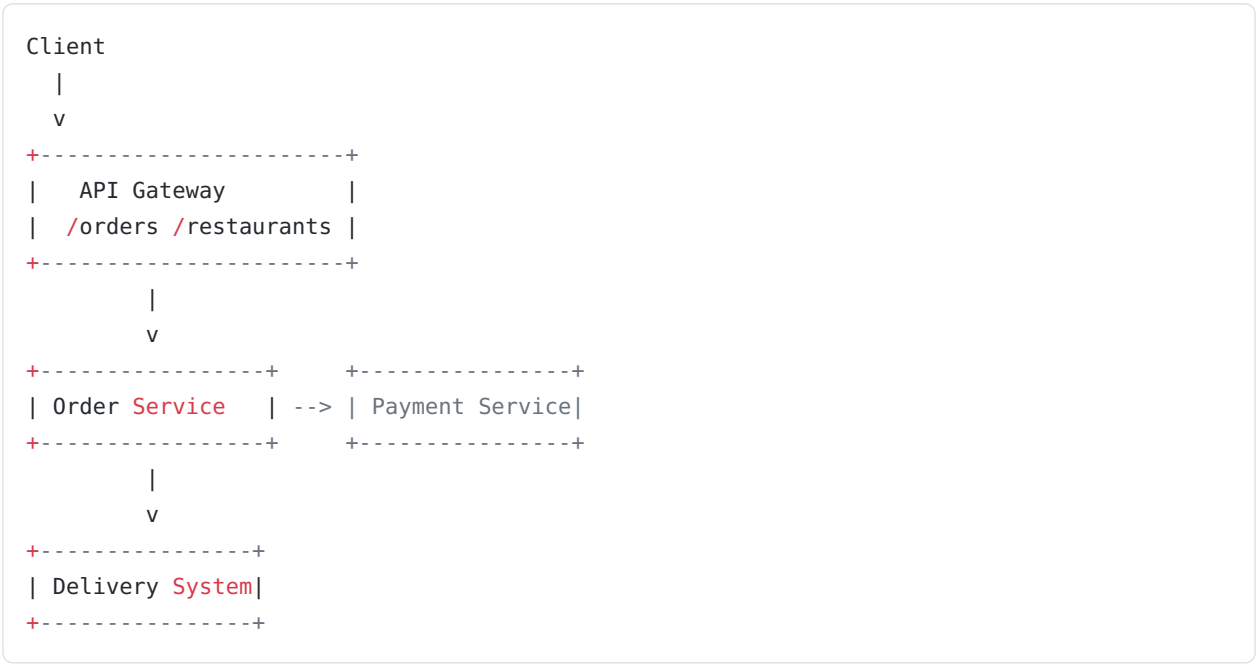
Even internally, **API principles still apply**:

- Clear method names
- Simple, consistent data contracts
- No side effects

- Backward compatibility for microservices

That's how you avoid breaking 10 downstream services when one team ships a change.

Visual Overview



Each box exposes **a clean API boundary**, allowing teams to work independently yet cohesively.

Try It Yourself

Sketch the API endpoints for a **Ride Sharing App** (like Uber):

- Think of entities like `Driver`, `Ride`, and `Rider`.
- What should your endpoints look like?

Look at your current project.

- Are your APIs consistent?
- Do error responses follow a uniform format?

Summary — What We Learned Today

- **APIs are contracts** — design them with empathy for the developer consuming them.
- **Consistency, clarity, and versioning** make APIs robust and evolvable.
- Use **proper nouns and HTTP methods** to express intent clearly.
- Always include **structured errors and pagination** for real-world scalability.

- API design is not just about REST — the same principles apply to **interfaces and service contracts** too.

Coming Up Next

Tomorrow, we'll **design a Chat System (WhatsApp/Slack style)** and apply these principles to real-time communication APIs — from message delivery to user presence tracking.

Follow the full series here: Low-Level Design in 25 Days: A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8) Save it and continue your journey toward becoming a design-savvy engineer.